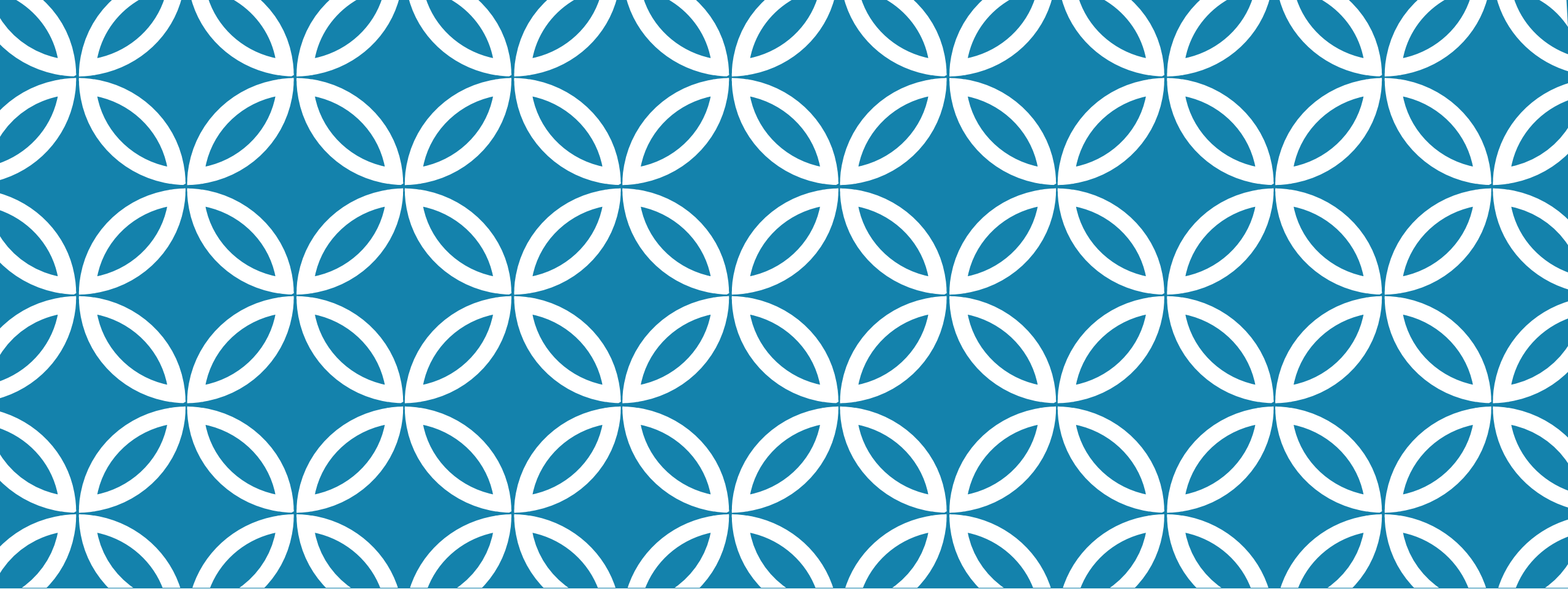




STRUCTURED QUERY LANGUAGE (SQL) AND MYSQL

SQL using MySQL

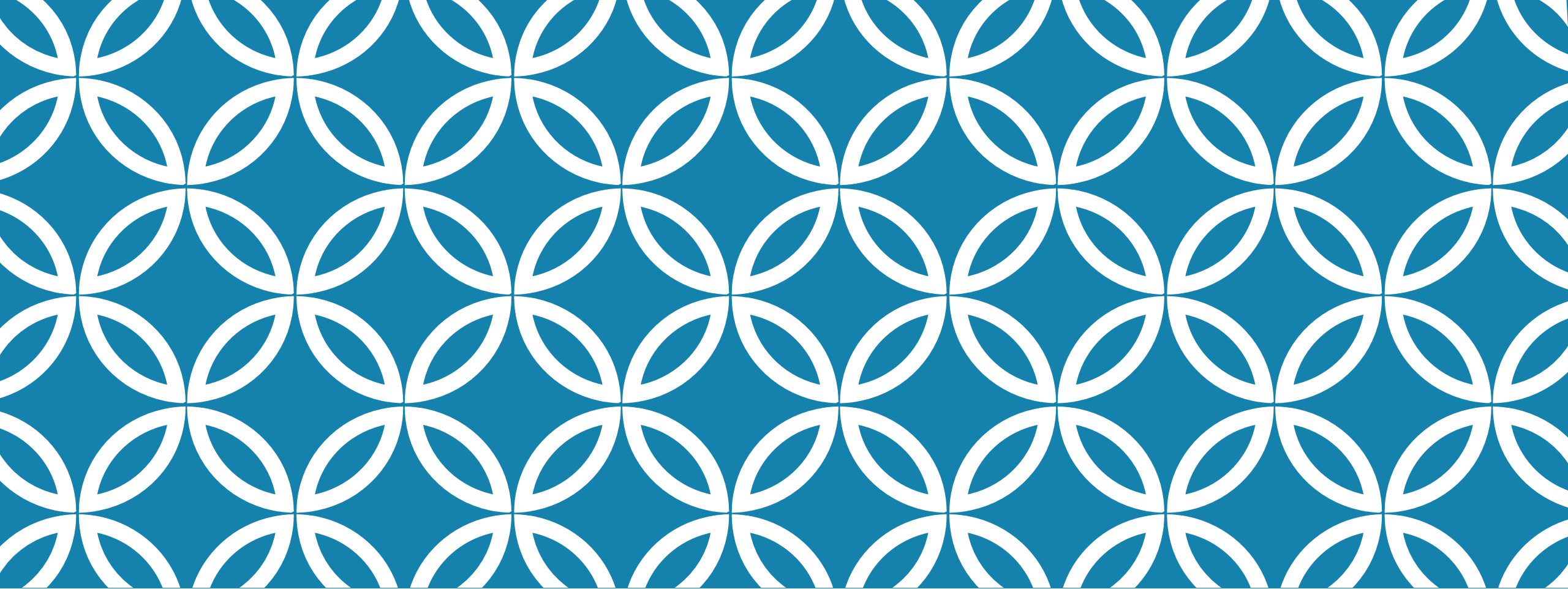
COPYRIGHT NOTICE

© PRASHANTH PURANIK, WWW.DIGDEEPER.IN

All rights reserved. No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the author, except in the case of brief quotations embodied in critical reviews and certain other non-commercial uses permitted by copyright law. For permission requests, write to the publisher, addressed "Attention:Permissions Coordinator" at the address below.

Email: puranik@digdeeper.in

Website: www.digdeeper.in



BEFORE WE BEGIN...

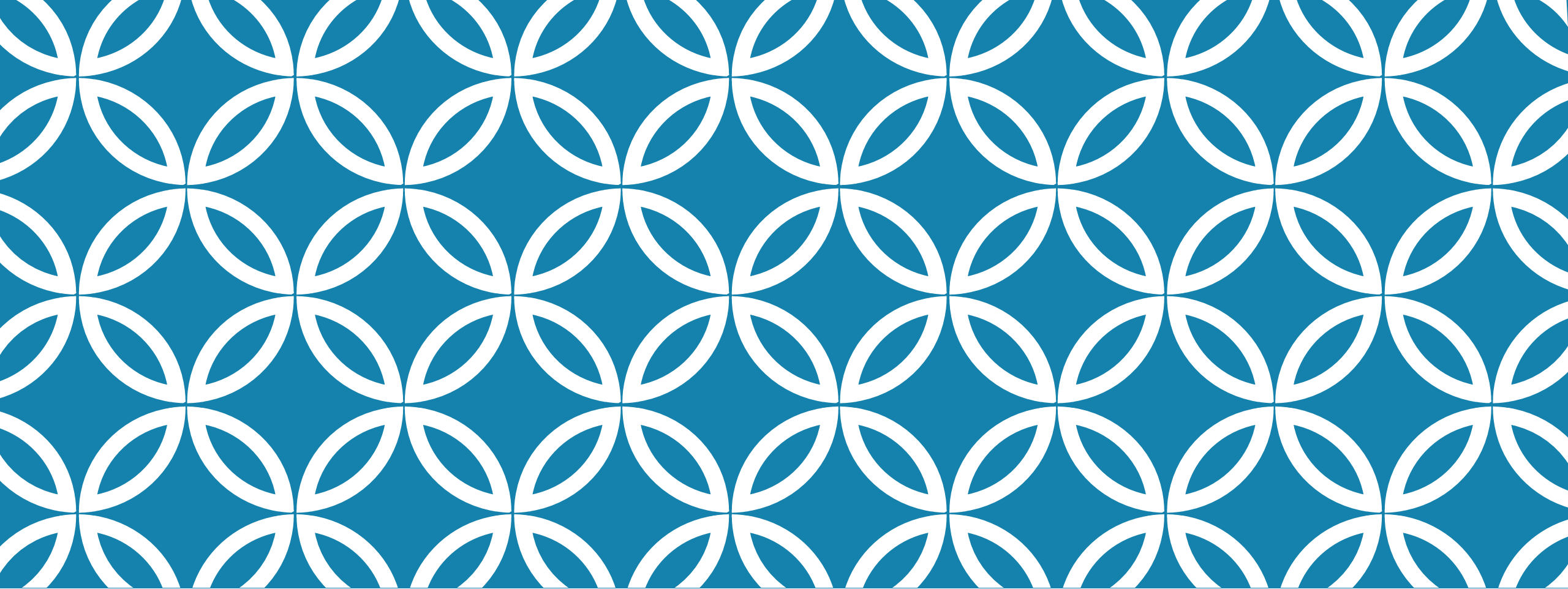
HARDWARE AND SOFTWARE
REQUIREMENTS

HARDWARE REQUIREMENTS

- Minimum 4GB (6GB or more recommended)
- Hard Disk: 128 GB or above
- CPU: Core i3 or above
- Operating System: Windows/Mac OSX/Linux
- Internet connection is required during training.

SOFTWARE REQUIREMENTS

- MySQL – Download and install the Community edition (free version) from <https://dev.mysql.com/downloads/mysql/>
- MySQL Workbench (GUI client) – Download and install from <https://dev.mysql.com/downloads/workbench/>



STRUCTURED QUERY LANGUAGE (SQL)

SQL using MySQL

SQL - WHAT AND WHY

Structured Query Language (SQL) is a special-purpose programming language

- To manipulate sets of data, typically from a relational database
- Became an ANSI standard in 1986 and ISO standard in 1987
- Most commands are specified in the standard, some are not – so different vendors will have same syntax for the most part, with some differences.

Note: Relational Database Management System (RDBMS) is only one way to describe data and the relationships between data entities. There are others like Document and Object DBMSes.

RDBMS

- Relational Database Management System (RDBMS) is a way to store data in tables
- Tables are also called relations. Hence the name Relational DBMS.
- Tables represent data of an entity in the real world. Example: A table to store data of contacts (contact being an entity).
- Tables have rows, where every row represents data of one instance (a particular contact, say John).
- Every row has columns, aka fields. A field has data of some aspect of the instance. Eg. The birth date of John.
- A database consists of a set of tables related to each other. Eg. contacts, emailids etc.

EXAMPLE: TABLE IN AN RDBMS

Table name: contacts

first_name	last_name	email
John	Doe	john.doe@example.com
Jane	Doe	jane.doe@example.com
Jane	Summers	jane.summers@example.com
Mark	Anthony	NULL

Query: What are all the contacts that have last name Doe?

NORMALIZATION – WHAT IT IS

Normalization is the process of restructuring a database (RDBMS) to reduce data redundancy and improve data integrity.

NORMALIZATION — WHY IS IT REQUIRED?

Consider this...

1. The previous schema leads to null values for email field

first_name	last_name	email
John	Doe	john.doe@example.com
Jane	Doe	jane.doe@example.com
Jane	Summers	jane.summers@example.com
Mark	Anthony	NULL

NORMALIZATION — WHY IS IT REQUIRED?

2. If we require to store multiple email ids, more null values would result (see below). Also, it would limit the maximum number of email ids that could be stored.

first_name	last_name	email1	email2
John	Doe	john.doe@example.com	john.doe@gmail.com
Jane	Doe	jane.doe@example.com	NULL
Jane	Summers	jane.summers@example.com	NULL
Mark	Anthony	NULL	NULL

NORMALIZATION – HOW

The previous problems are eliminated when we store the person details (say, contacts) in one table and email ids for the contacts in a separate table (say, emails)

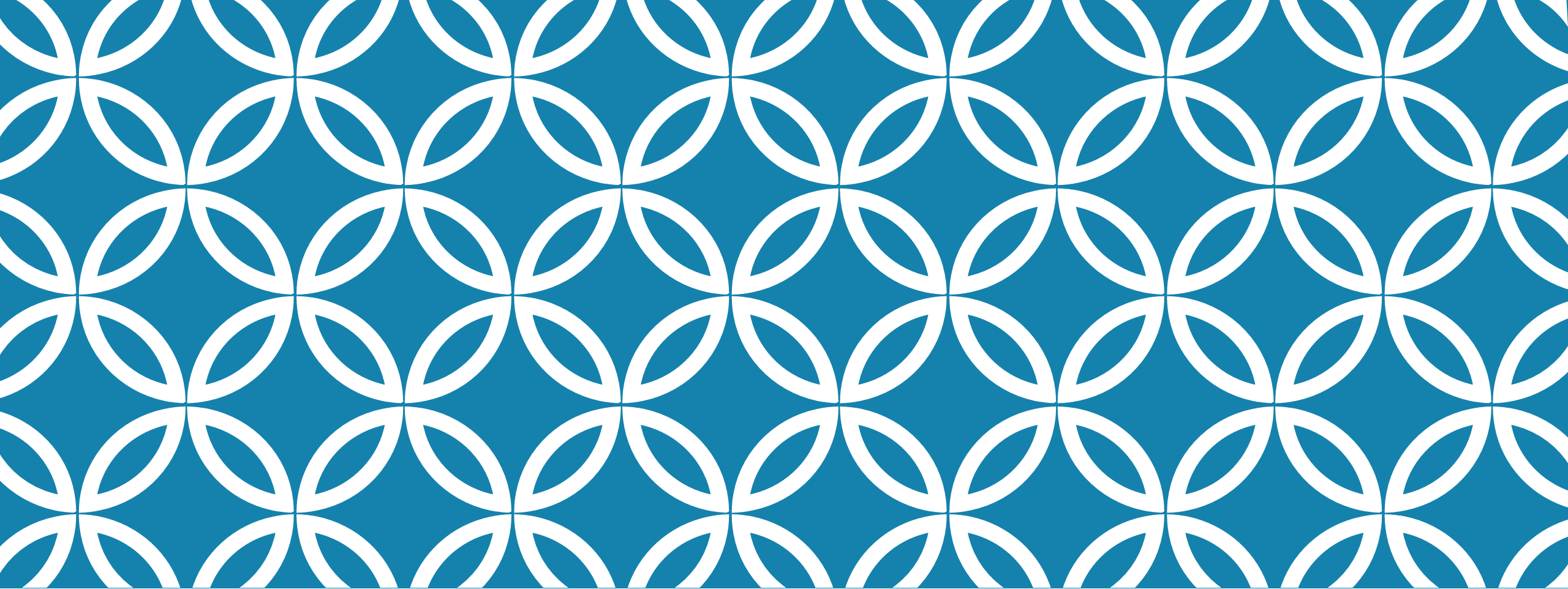
id	first_name	last_name
1	John	Doe
2	Jane	Doe
3	Jane	Summers
4	Mark	Anthony

id	email
1	<u>john.doe@example.com</u>
1	<u>john.doe@gmail.com</u>
2	<u>jane.doe@example.com</u>
3	<u>jane.summers@example.com</u>

A NOTE ON DB DESIGN

Database design is important; it controls the questions you can ask later.

SQL is the language you use to ask the questions.

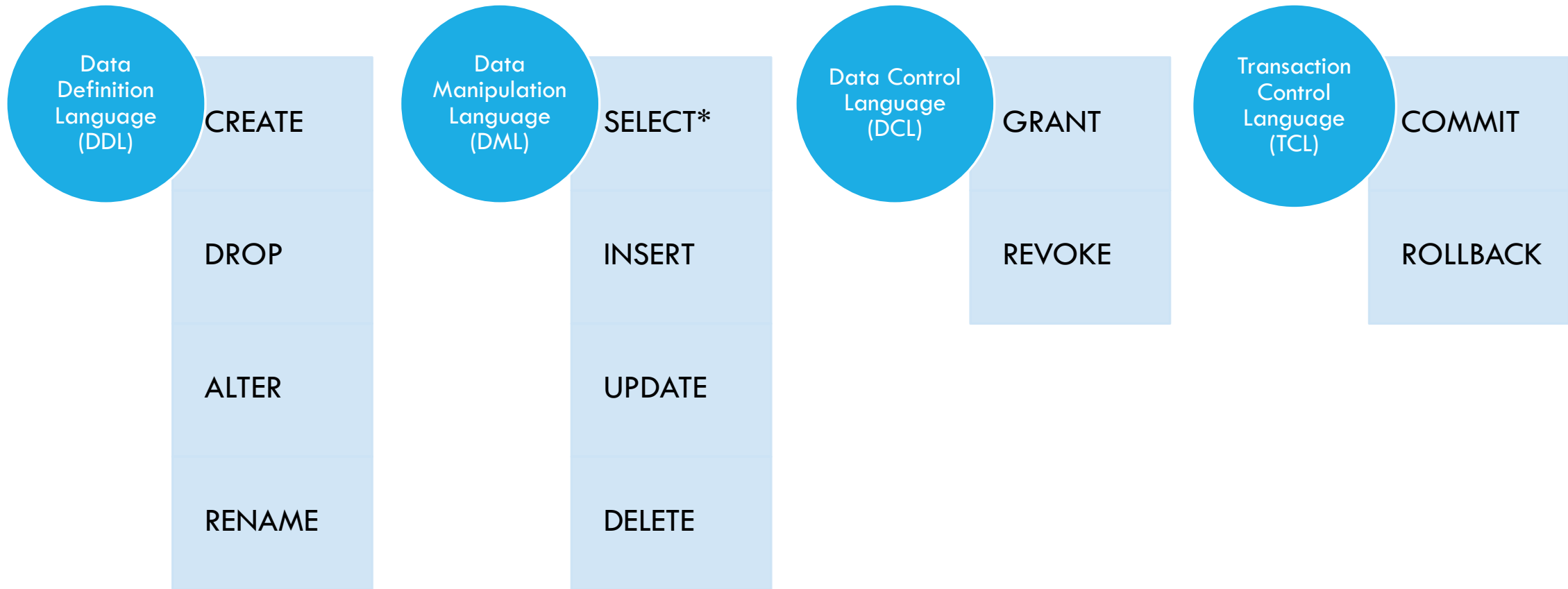


CLASSIFICATION OF SQL STATEMENTS

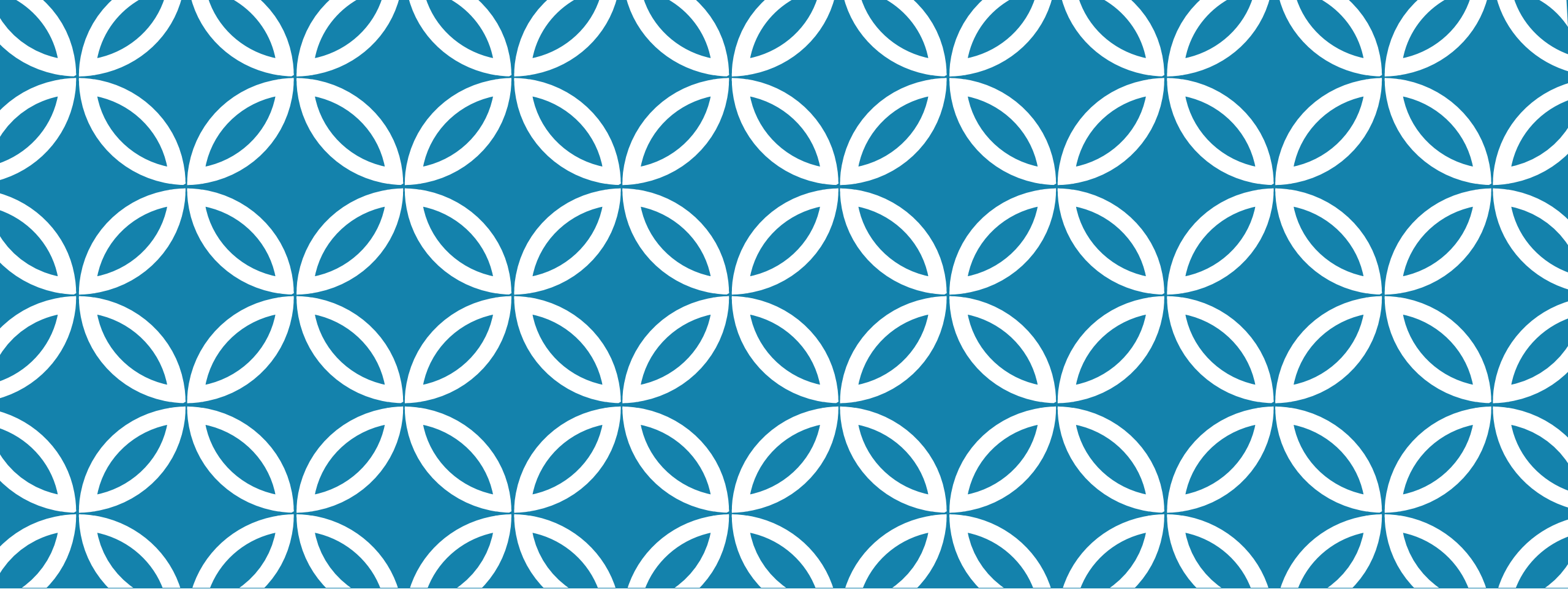
SQL STATEMENT — WHAT IT IS

An SQL Statement
is an expression that
tells a database
what you want it to do.

CLASSIFICATION OF SQL STATEMENTS



* The SELECT statement is sometimes classified as being part of Data Query Language (DQL), as it does not modify any data.



MYSQL

ABOUT MySQL

MYSQL

- Some popular RDBMSes are MySQL, Oracle SQL Server, Microsoft SQL Server, PostgreSQL
- MySQL is an open-source RDBMS
- Part of the LAMP/WAMP/MAMP stack – others being Linux/Windows/Mac OS, Apache server and PHP/Perl/Python
- MySQL users include Google and Facebook
- Available for Windows, Linux and Mac OS etc.
- Has been around for more than 20 years

MYSQL — GETTING STARTED

- To launch the MySQL server and connect using MySQL client, follow the steps mentioned here

<https://dev.mysql.com/doc/mysql-getting-started/en/>

- You will need to know

- Location where MySQL is installed
- The **user name** (usually **root**) and **password** (this would be setup by the one who installed MySQL)

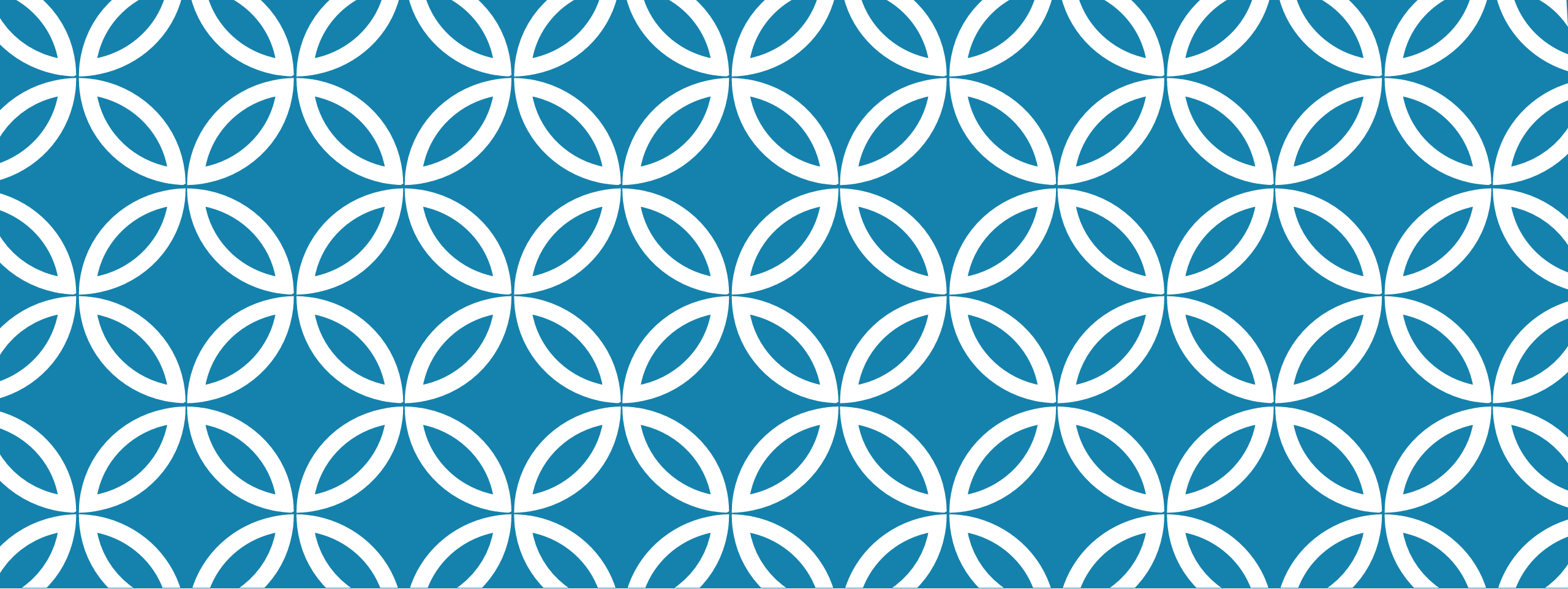
- You should start the MySQL Server (if not already running)

<https://stackoverflow.com/questions/1509148/how-to-start-mysql-server-on-windows-xp>

- To start CLI client (you will be prompted for password)

```
$> mysql -u root -p
```

- We will use MySQL Workbench (the GUI client) in the training



BEFORE GETTING STARTED WITH QUERIES

SQL COMMENTS

➤ Multi-line comments

/*

This comment spans
multiple lines

*/

➤ Single-line comment

-- This is a comment which CANNOT span multiple lines

SQL STRING LITERALS

- String literals may be single-quoted or double-quoted

`'hello'`

`"world"`

- Special characters have escape sequences

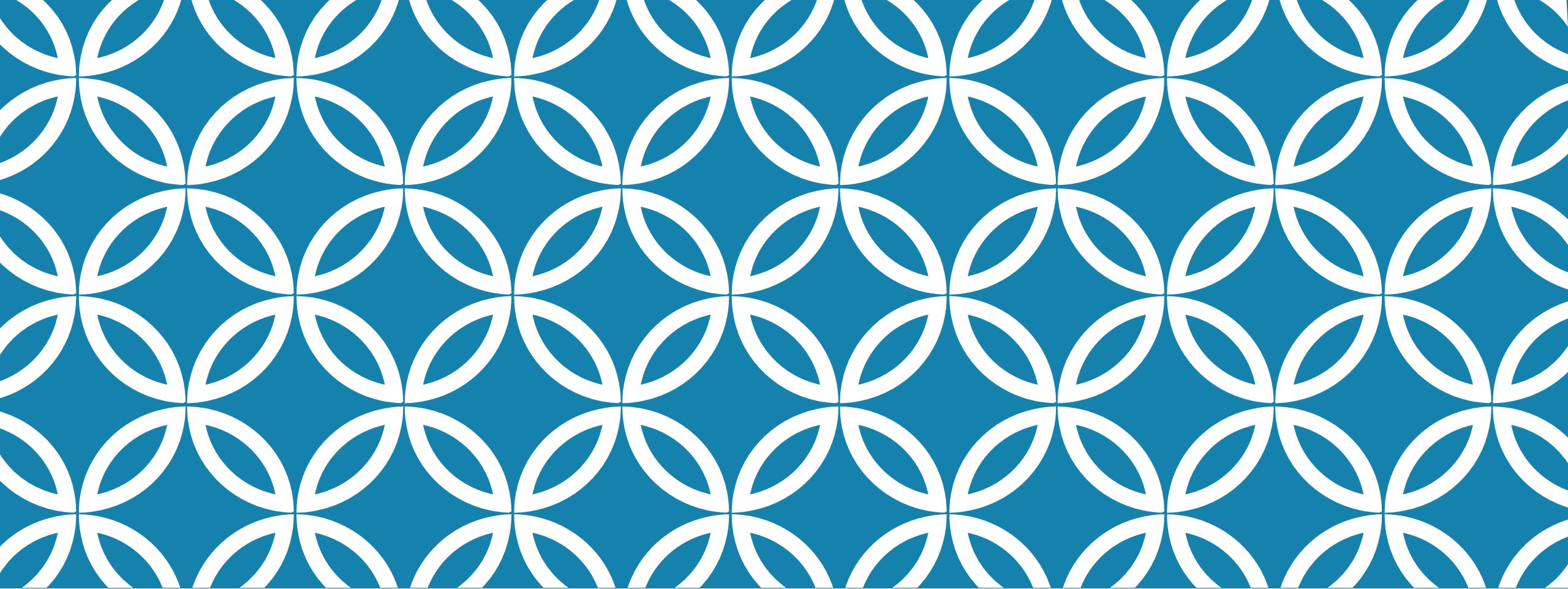
- `'it\'s a beautiful day'` – here the `\'` is a single character representing a single-quote

- Some other escape sequences are `\`, `\n`, `\%`, `\_`

- **Note:** `%` and `_` are used as wildcard characters in string matching

- Apart from these, in order to work with tables, databases etc. whose names may have special characters, those names are enclosed in backticks (```). Backticks cannot be used for string literals.

`SELECT * from `expedited-orders`;`



BASIC SQL SYNTAX (DML)

Create-Read-Update-Delete
(CRUD)

BASIC SQL STATEMENTS — SELECT

- `SELECT first_name FROM contacts;*`
- `SELECT first_name, last_name FROM contacts;`

In general...

- `SELECT col1, col2, ... FROM table`
- `SELECT * FROM table;`
- **Note:** Multiple tables can be queried as we shall see later

* SQL commands are not case-sensitive – SELECT is same as Select/select etc. Table names may or may not be case-sensitive (Windows/Mac OS they are not, in Linux they are)

BASIC SQL STATEMENTS — INSERT

- `INSERT INTO contacts (first_name) VALUES ("Barry");`
- `INSERT INTO contacts (first_name, last_name) VALUES ("Barry", "Brown");`

In general...

- `INSERT INTO table (col1, col2, ...) VALUES (val1, val2, ...);`
- `INSERT INTO table VALUES (val1, val2, ...);`

BASIC SQL STATEMENTS — UPDATE

- UPDATE contacts SET last_name = “Spencer” WHERE first_name = “Barry”;
- UPDATE contacts SET last_name = “Spencer”

In general...

- UPDATE table
SET col1 = val1, col2 = val2, ...
WHERE <condition>;

BASIC SQL STATEMENTS — DELETE

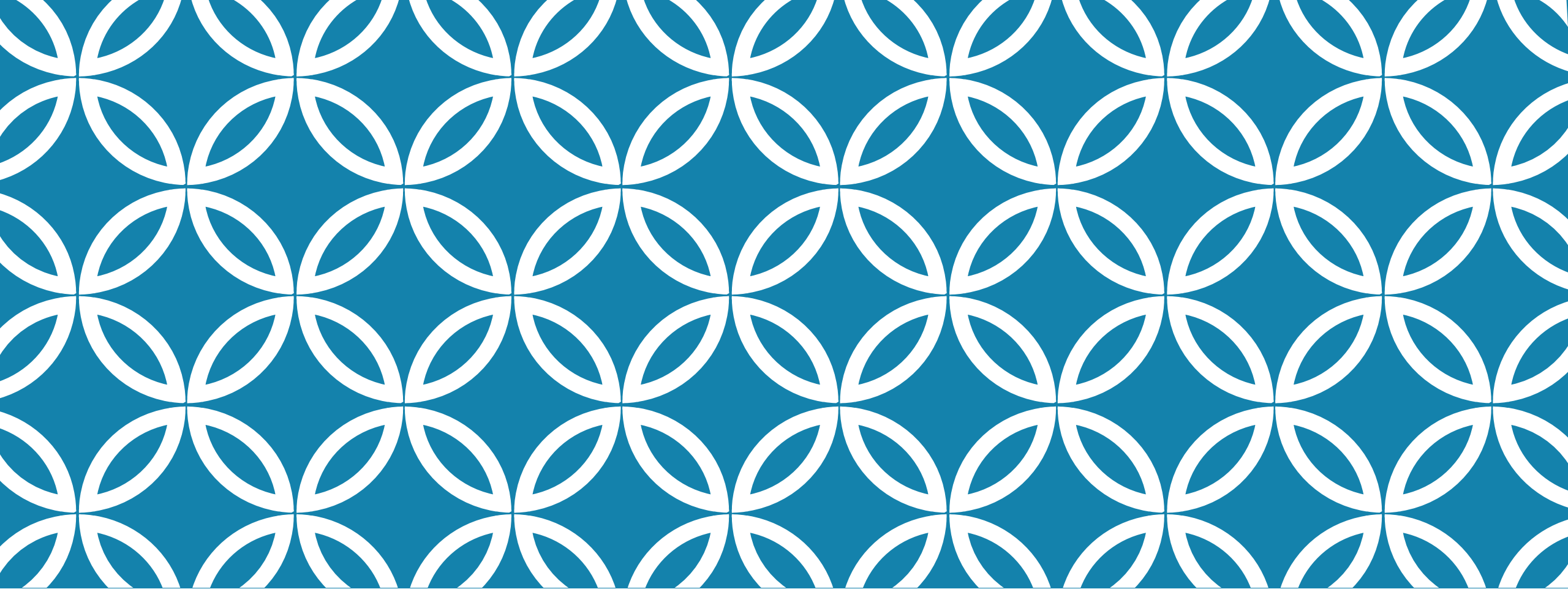
➤ `DELETE from contacts WHERE id = 2;`

In general...

➤ `DELETE from table
WHERE <condition>;`

➤ `DELETE from contacts;`

WARNING: You probably should NEVER do this!!



DETAILED SQL SYNTAX (DML)

SELECT

SELECT

- **SELECT** basically answers questions you have
 - Who are all my contacts?
 - Who are the contacts with first name Jane?
 - How many contacts do I have?

SELECT — HARD-CODED SELECTION

➤ A hard-coded selection

```
SELECT val1, val2, ...;
```

```
SELECT 'hello', 'world';
```

```
SELECT 'hello' AS col1, 'world' AS col2;
```

SELECT — NORMAL USAGE

- Selecting a set of columns from a table

```
SELECT col1, col2, ... FROM table;
```

```
SELECT first_name, last_name FROM contacts;
```

```
SELECT first_name AS fname, last_name AS lname  
FROM contacts;
```

SELECT — TABLE NAME ALIAS

- Qualify the table name in column list specification — is clearer, allows auto-completion, and can be efficient (when using multiple tables in query)

```
SELECT contacts.first_name, contacts.last_name FROM contacts;
```

- Alias for the table name can also be provided

```
SELECT c.first_name, c.last_name FROM contacts c;
```

SELECT -THE * WILDCARD

- Selecting all columns from a table

```
SELECT * FROM table;
```

- **Tip:** If you are not actually using many column values returned by the query, then explicitly select columns.

SELECT — DISTINCT AND NOT DISTINCT CLAUSES

- DISTINCT is used to select **unique** rows from table

```
SELECT first_name FROM contacts;
```

vs

```
SELECT DISTINCT first_name FROM contacts;
```

```
SELECT first_name, age FROM contacts;
```

vs

```
SELECT DISTINCT first_name, age FROM contacts;
```

SELECT — WHERE CLAUSE

- Selects a subset of rows

```
SELECT * FROM table WHERE <condition>;
```

```
SELECT last_name FROM contacts WHERE first_name =  
'Jane';
```

- **Note:** IN SQL, order of clauses is to be maintained.
WHERE comes **after** FROM

DETOUR : BOOLEAN OPERATORS

- Conditions in SQL can use any of these Boolean operators
- Additionally, the NOT operator is used to negate conditions

Operator	Condition it tests
=	Equals
<>	Not Equals
>	Greater than
>=	Greater than or Equals
<	Less than
<=	Less than or equals
BETWEEN	Value in the range (includes the bounds)
LIKE	Value matches pattern (uses %, _ in strings)
IN	Value in list
IS	Similar to Equals but only for NULL check
IS NOT	Similar to Not Equals but only for NULL check

DETOUR : BOOLEAN OPERATORS

- ...WHERE age BETWEEN 18 and 65
- ...WHERE first_name LIKE 'J%'
- ...WHERE last_name LIKE 'D_ _'
- ...WHERE last_name LIKE '%e%'
- ...WHERE last_name IN ('Doe', 'Summers')
- ...WHERE age IS NULL
- ...WHERE age IS NOT NULL

DETOUR : COMBINING INDIVIDUAL CONDITIONS

➤ AND keyword

```
SELECT * FROM contacts  
WHERE first_name = 'Jane' AND age <= 30
```

➤ OR keyword

```
SELECT * FROM contacts  
WHERE first_name = 'Jane' OR age <= 30
```

SELECT — ORDER BY CLAUSE

- Sorts the selected rows

```
SELECT * FROM table WHERE <condition> ORDER BY <condition>;
```

```
SELECT * FROM table ORDER BY <condition>;
```

- Example

```
SELECT * FROM contacts ORDER BY first_name
```

- Condition can include multiple comma-separated list of columns. The sort happens on first column, ties are broken by sorting on second column, and so on.

```
SELECT * FROM contacts ORDER BY first_name, last_name
```

SELECT — ORDER BY CLAUSE

- Sorts the selected rows

```
SELECT * FROM table WHERE <condition> ORDER BY  
<condition>;
```

```
SELECT * FROM table ORDER BY <condition>;
```

- Example

```
SELECT * FROM contacts ORDER BY first_name
```

SELECT — ORDER BY CLAUSE

- Condition can include multiple comma-separated list of columns. The sort happens on first column, ties are broken by sorting on second column, and so on.

```
SELECT * FROM contacts ORDER BY first_name, last_name
```

- Sort order is ascending (by default), but can be made descending

```
SELECT * FROM contacts ORDER BY first_name ASC,  
last_name DESC
```

SELECT — AGGREGATE FUNCTIONS

- Computes new values from a column's values
- Used in place of column in select list
- Take column name as input, and output an aggregate value
- Used with **DISTINCT** to get aggregate unique values of column

Function	Description
COUNT	Count of the column specified (includes NULL values only if * is used)
MAX	Maximum value in the column
MIN	Minimum value in the column
AVG	Average of all values in the column (can be used only for numeric)
SUM	Sum of all values in the column (can be used only for numeric)

SELECT — AGGREGATE FUNCTIONS

- `SELECT COUNT(last_name) FROM contacts;`
- `SELECT MAX(age) FROM contacts;`
- `SELECT AVG(age) FROM contacts;`
- `SELECT AVG(age) AS AVERAGE_AGE FROM contacts;`
- `SELECT AVG(age) AS AVERAGE_AGE_OF_DOES FROM contacts WHERE last_name = 'Doe';`
- `SELECT COUNT(DISTINCT last_name) FROM contacts;`

SELECT — GROUP BY CLAUSE

- Partitions the rows into subsets, thus forming groups of rows
- Aggregate functions can also be computed on the subsets, thus forming 1 row for every subset
- However, column used in GROUP BY must also appear in select list
- Appears after WHERE clause (if exists)

```
SELECT last_name, COUNT(last_name) FROM contacts GROUP BY last_name;
```

SELECT — HAVING CLAUSE

- Works against GROUP BY clause, the way WHERE works against SELECT
- Filters the subsets created by GROUP BY clause

```
SELECT last_name, COUNT(*)  
FROM contacts  
GROUP BY last_name  
HAVING COUNT(*) >= 2;
```

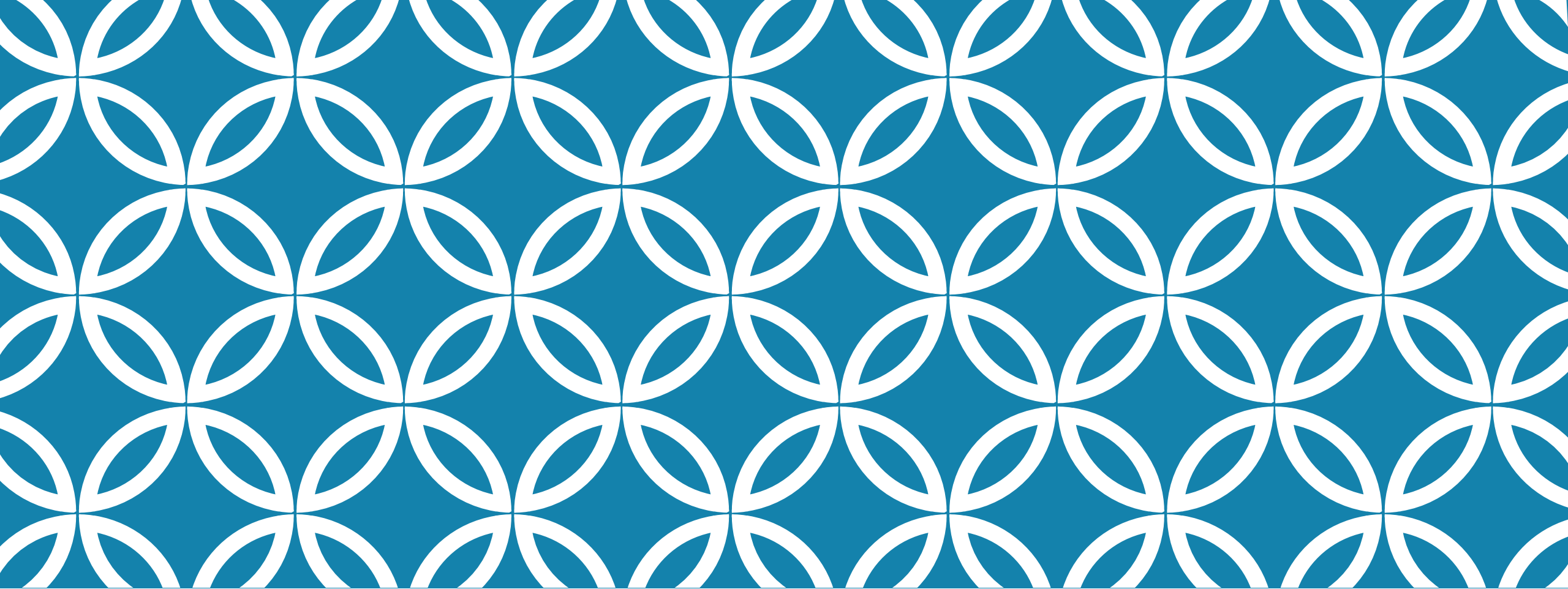
- Alternatively, using an alias

```
SELECT last_name, COUNT(last_name) AS last_name_count  
FROM contacts  
GROUP BY last_name  
HAVING last_name_count >= 2;
```

SELECT — LIMIT CLAUSE

- You can limit the number of rows retrieved using this
SELECT column1, column2, ...
FROM table
LIMIT offset , count;
- For example to start from the 101st row* and retrieve 10 rows,
SELECT *
FROM contacts
LIMIT 100, 10
- This clause helps implement pagination in applications

* The offset of the first row is 0 and not 1



DETAILED SQL SYNTAX (DML)

SELECT - USING JOINS TO
RETRIEVE DATA FROM MULTIPLE
TABLES

JOINS — COMBINING DATA FROM MANY TABLES

- Select data from multiple tables using joins
- WHERE clause is typically included
- WHERE clause filters rows included from each table, and provides conditions to join rows from different tables
- There are different types of joins

TABLE STRUCTURE

Consider the following structure for rest of the section on joins.

id	first_name	last_name
1	John	Doe
2	Jane	Doe
3	Jane	Summers
4	Mark	Anthony

id	email
1	<u>john.doe@example.com</u>
1	<u>john.doe@gmail.com</u>
2	<u>jane.doe@example.com</u>
3	<u>jane.summers@example.com</u>
5	<u>dorothy.summers@example.com</u>

CROSS JOIN

- Cross product (Cartesian product) of rows from tables
- No WHERE clause
- Not very useful, and not sensible in most cases

```
SELECT * FROM contacts, emails;
```

INNER JOIN

- Most typical join
- Most common usage relates primary key in one table to corresponding foreign key in another table, using a condition.

```
SELECT *
```

```
FROM contacts c INNER JOIN emailids e ON c.id = e.contact_id;
```

- Does not have rows from either table that have no match

LEFT OUTER JOIN

- Has rows from table to left of JOIN clause that have no match in the table to right of the clause
- NULL values for unmatched row's column values for the right side table.

```
SELECT *
```

```
FROM contacts c LEFT OUTER JOIN emailids e ON c.id = e.contact_id;
```

RIGHT OUTER JOIN

- Has rows from table to right of JOIN clause that have no match in the table to left of the clause
- NULL values for unmatched row's column values for the left side table.

```
SELECT *
```

```
FROM contacts c RIGHT OUTER JOIN emailids e ON c.id = e.contact_id;
```

FULL OUTER JOIN

- Has rows from either table that have no match – NULL values for column values for these

```
SELECT *
```

```
FROM contacts c FULL OUTER JOIN emailids e ON c.id = e.contact_id;
```

- MySQL does not support full outer join. They can be emulated using the UNION operator

```
SELECT *
```

```
FROM contacts c LEFT OUTER JOIN emailids e ON c.id = e.contact_id
```

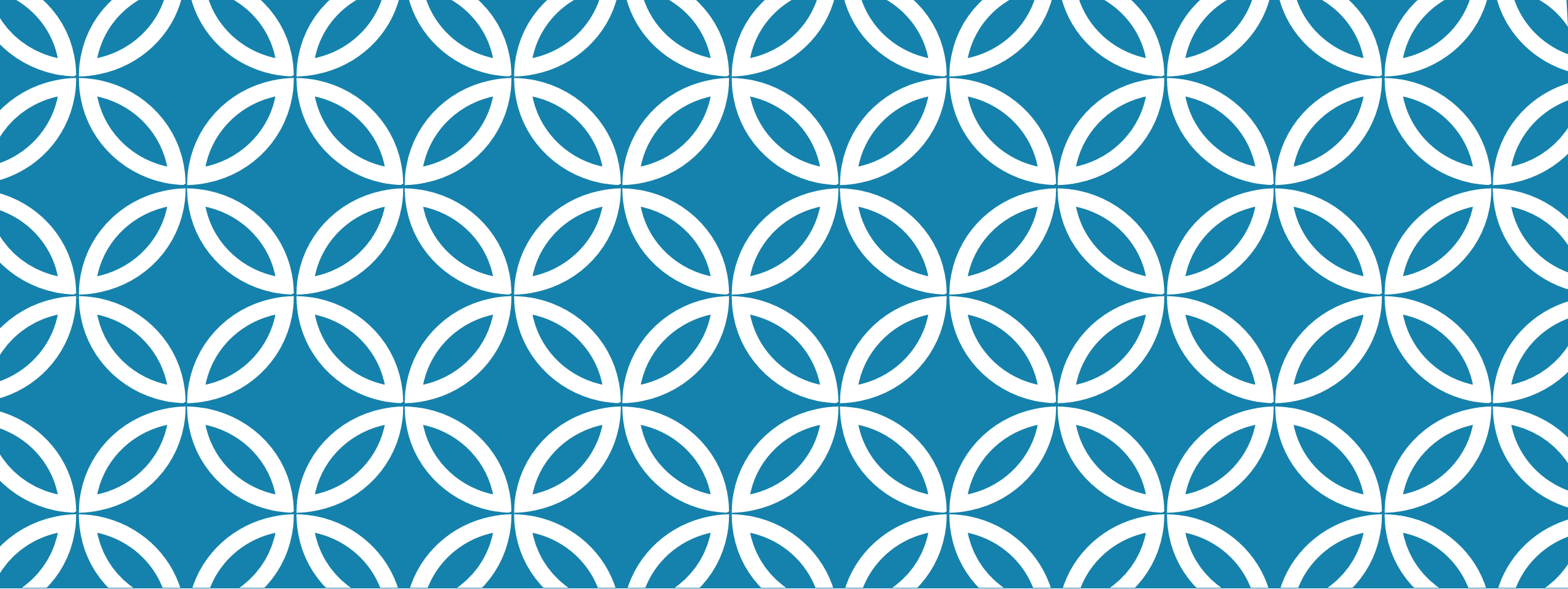
```
UNION
```

```
SELECT *
```

```
FROM contacts c RIGHT OUTER JOIN emailids e ON c.id = e.contact_id;
```

SELF JOIN

- Joins a table to itself
- Usually used when a column in a table references values from another (or same) column in the same table
- Arises for example in hierarchical data
- Example: An employee table where manager_id refers to id column from the table



DETAILED SQL SYNTAX (DML)

INSERT

INSERT

➤ Can insert records into only one table at a time

➤ In general...

```
INSERT INTO table (col1, col2, ...) VALUES (val1, val2, ...);
```

```
INSERT INTO table VALUES (val1, val2, ...);
```

➤ Example

```
INSERT INTO contacts(first_name, last_name) VALUES ( 'John', 'Doe', 32);
```

```
INSERT INTO contacts(first_name, last_name) VALUES ( 'Philip', NULL, 28);
```

BULK INSERT

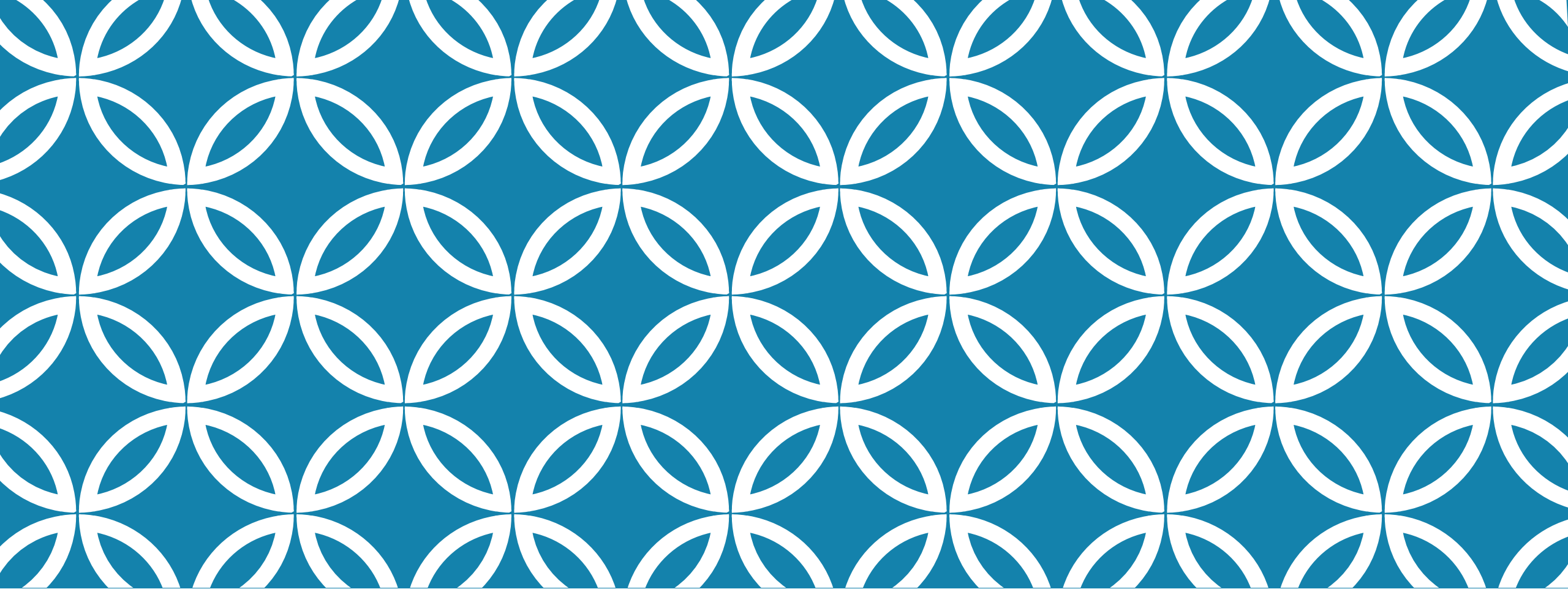
- Multiple rows can be created simultaneously

```
INSERT INTO contacts VALUES ('Jennifer', 'Garner', 5), ('Tony', 'Stark', 6);
```

- You can also select rows from another query and insert those

```
INSERT INTO contacts_family
```

```
SELECT * FROM contacts where last_name = 'Doe';
```



DETAILED SQL SYNTAX (DML)

UPDATE

UPDATE

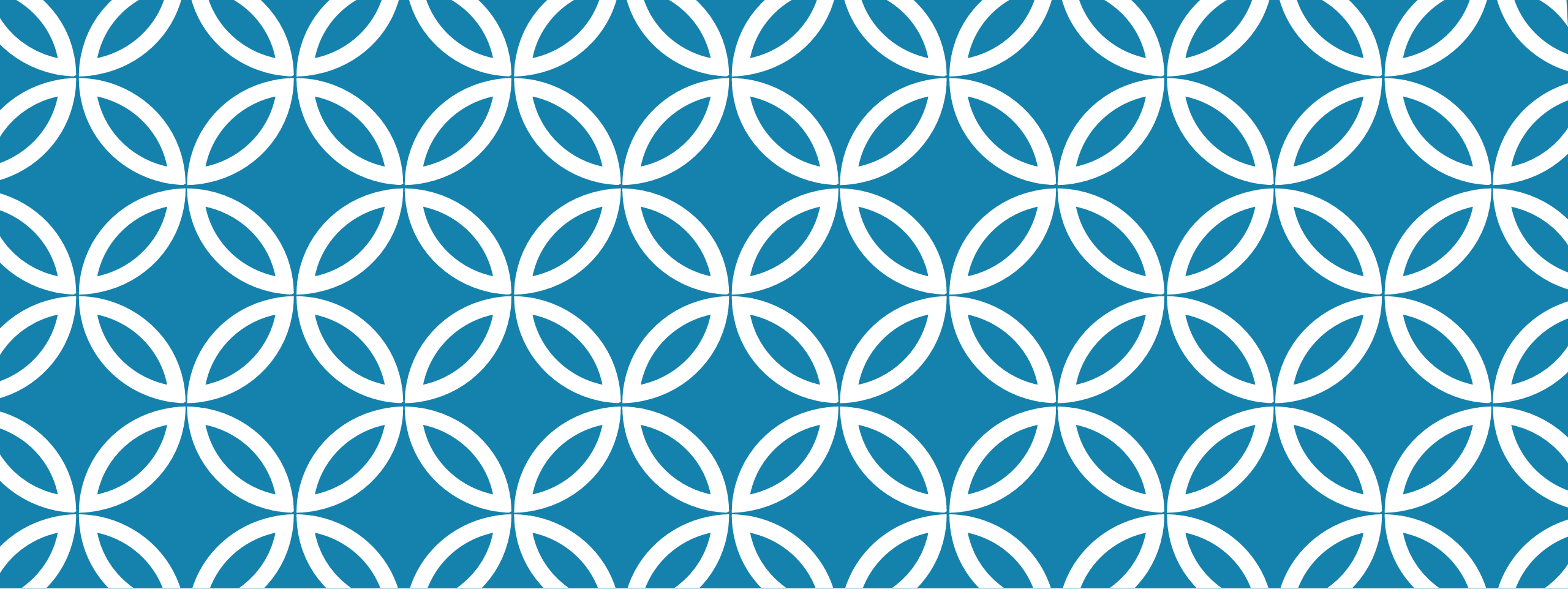
- Can update records of only one table at a time
- The WHERE clause is not necessary, but is a bad idea to omit it
- SQL Workbench turns on safe update mode by default which prevents update without a WHERE clause having a key column*
- In general...

UPDATE table

SET col1 = val1, col2 = val2, ...

WHERE <condition>;

* <https://stackoverflow.com/questions/11448068/mysql-error-code-1175-during-update-in-mysql-workbench>



DETAILED SQL SYNTAX (DML)

DELETE

DELETE

- Can delete records of only one table at a time
- Deletion is permanent!
- The WHERE clause is not necessary, but is a bad idea to omit it
- SQL Workbench turns on safe update mode by default which prevents delete without a WHERE clause having a key column* This is ok though!

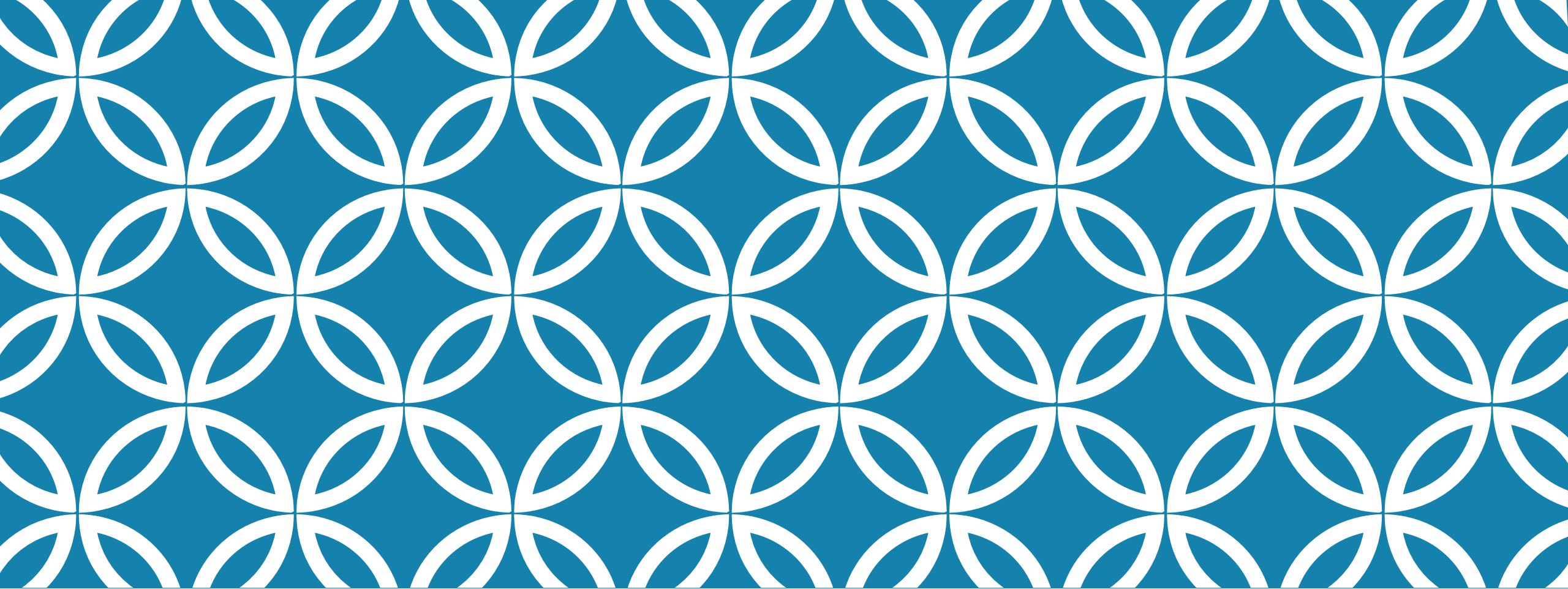
DELETE FROM contacts WHERE id > 0;

- In general...

DELETE from table

WHERE <condition>;

* <https://stackoverflow.com/questions/11448068/mysql-error-code-1175-during-update-in-mysql-workbench>



DDL

CREATING DATABASES AND TABLES

DDL STATEMENTS

- GUI tools have interface to create databases and tables
- However we may need to do these tasks at the application level
- So it is better to know DDL statements

USE DATABASE

- The USE command scopes future queries to a DB

```
USE <database_name>;
```

- Example

```
USE storedb;
```

```
SELECT * FROM products ...
```

- Without the use command, we would need to scope the table to the DB explicitly

```
SELECT * FROM storedb.products ...
```

CREATE DATABASE

- Not part of the SQL standard – but it is similar across DBMSes
- Syntax and example

```
CREATE DATABASE <db_name>;
```

```
CREATE DATABASE storedb;
```

DROP DATABASE

- Not part of the SQL standard – but it is similar across DBMSes
- Removes the database entirely
- Syntax and example

`DROP DATABASE <db_name>;`

`DROP DATABASE storedb;`

CREATE TABLE - SQL DATA TYPES

➤ Data types affect the kind of data that can be stored in a column

Data Type	Value Space
CHAR, NCHAR*	Can hold N character values - set to N statically
VARCHAR, NVARCHAR*	Can hold N character values - set to N dynamically – storage can be less than N
BINARY	Hexadecimal data
SMALLINT	-2^{15} to $2^{15} - 1$
INTEGER	-2^{31} to $2^{31} - 1$
BIGINT	-2^{63} to $2^{63} - 1$
BOOLEAN	TRUE and FALSE
DATE	YEAR, MONTH and DAY in the format YYYY-MM-DD
TIME	HOUR, MINUTE and SECOND in the format HH:MM:SS[.sF]
TIMESTAMP	Both DATE and TIME

* NCHAR and NVARCHAR can store Unicode characters

CREATE TABLE

- Is part of the SQL standard
- Simplest syntax

```
CREATE TABLE <table_name> ( col1_name col1_type, col2_name, col2_type, ... )
```

```
CREATE TABLE contacts
```

```
(
```

```
    first_name varchar(40),
```

```
    last_name varchar(40),
```

```
    age int,
```

```
    id int
```

```
);
```

DROP TABLE

- Is part of the SQL standard
- Removes a table from the database
- Simplest syntax

`DROP TABLE <table_name>;`

- In MySQL you can drop multiple tables in one go

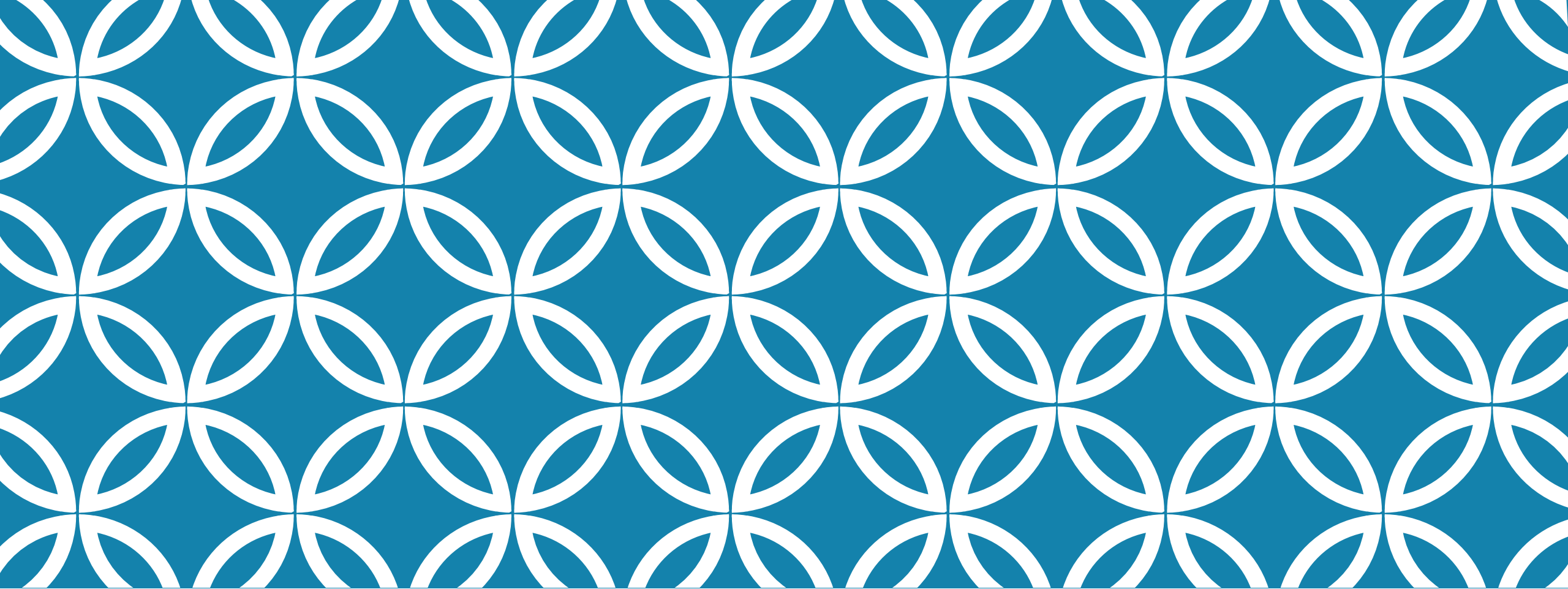
`DROP TABLE <table_1_name>, <table_2_name>, ...;`

TRUNCATE TABLE

- Is part of the SQL standard
- It removes all the rows from the table
- Syntax

TRUNCATE TABLE <table_name>;

- Unlike DELETE * FROM <table_name>, the truncate operation cannot be rolled back when part of a transaction.



COLUMN CONSTRAINTS

CONSTRAINTS ON VALUES
THAT CAN BE STORED IN A
COLUMN

COLUMN CONSTRAINTS

- Column constraints are restrictions placed on a column
- Examples
 - A value **MUST** be present in a column – eg. first_name in contacts table
 - A value must be unique in a column – eg. email must be unique in contacts table
- They further help enforce data integrity
- They are specified as a space separated list, after the data type for column, when creating tables

```
CREATE TABLE <table_name> ( col1_name col1_type constraint1 constraint2 ..., ... )
```

NULL VALUES

- NULL is a special value in SQL (it is a keyword)
- Indicates a missing value
- Column value may be required, or not required
- Required columns cannot have missing values and hence, constrained to be NOT NULL
- By default columns are **not** constrained to be NOT NULL – hence they can have missing values

CREATE TABLE - NOT NULL

```
CREATE TABLE contacts
```

```
(
```

```
    first_name varchar(40) NOT NULL,
```

```
    last_name varchar(40),
```

```
    age int,
```

```
    id int NOT NULL
```

```
);
```

PRIMARY KEY

- A unique value in a row that (uniquely) identifies the row
- Can also be composite - a combination of column values
- Primary key columns cannot be NULL (i.e. no missing values)
- A table can have **only one** primary key
- Examples:
 - email in a contacts table
 - apartment_address and flat_number in an address table

CREATE TABLE - PRIMARY KEY (COLUMN LEVEL)

```
CREATE TABLE contacts
```

```
(
```

```
    id int PRIMARY KEY
```

```
    first_name varchar(40) NOT NULL,
```

```
    last_name varchar(40),
```

```
    age int
```

```
);
```

CREATE TABLE - PRIMARY KEY (TABLE LEVEL)

```
CREATE TABLE contacts
(
    id int
    first_name varchar(40) NOT NULL,
    last_name varchar(40) NOT NULL,
    age int,
    PRIMARY KEY (first_name, last_name)
);
```

ALTER TABLE

- You can change a table's schema in the following ways
 - Add a column
 - Remove a column
 - Rename a column
 - Change the data type of a column (causes data to be converted)
 - Add a constraint
 - Remove a constraint
 - ...
- Left as self-study 😊

Reference: <https://www.tutorialspoint.com/sql/sql-alter-command.htm>

UNIQUE

- Imposes the constraint that column value is unique for every row
- You can also constrain a combination of column values to be unique
- Examples:
 - email in a contacts table
 - Aadhaar number in person table

CREATE TABLE - UNIQUE (COLUMN LEVEL)

```
CREATE TABLE person
(
    first_name varchar(40) NOT NULL,
    last_name varchar(40),
    aadhaar_number VARCHAR(12) UNIQUE,
    id int PRIMARY KEY
);
```

CREATE TABLE - UNIQUE (TABLE LEVEL)

```
CREATE TABLE person
(  
    first_name varchar(40) NOT NULL,  
    last_name varchar(40),  
    aadhaar_number VARCHAR(12) UNIQUE,  
    id int PRIMARY KEY,  
    UNIQUE (first_name, last_name)  
);
```

AUTO INCREMENT

- Sets up automatically generated sequence of numbers to be filled in as rows get inserted
- Useful to generate a column with unique values, especially as a primary key
- Examples:
 - id in a contacts table
- The keyword differs across DBMSes*. In MySQL it is AUTO_INCREMENT
- Note: Oracle SQL Server uses a command known as CREATE SEQUENCE to create a named sequence generator

* https://www.w3schools.com/sql/sql_autoincrement.asp

CREATE TABLE - AUTO_INCREMENT

```
CREATE TABLE contacts
```

```
(
```

```
    id int AUTO_INCREMENT PRIMARY KEY,
```

```
    first_name varchar(40) NOT NULL,
```

```
    last_name varchar(40),
```

```
    age int
```

```
);
```

➤ To start from a value other than 1 use

```
ALTER TABLE contacts AUTO_INCREMENT = 100;
```

FOREIGN KEY

- A value in a row that is a primary key in another table
- They represent the relations between tables in an RDBMS
- Examples:
 - address_id in a persons table exists as the primary key id in the addresses table
 - base_location in employees table exists as the primary key location_id in the offices table
 - customer_number in orders table exists as the primary key customer_number in the customers table
- Checks are performed during CRUD operations in order that data integrity is maintained. This is called **referential integrity**.
- Example: You cannot insert an employee with base_location if it does not appear as the location_id value for some row in offices table

CREATE TABLE - FOREIGN KEY

```
CREATE TABLE categories (  
    cat_id int not null auto_increment primary key,  
    cat_name varchar(255) not null,  
    cat_description text  
)
```

```
CREATE TABLE products (  
    prd_id int not null auto_increment primary key,  
    prd_name varchar(355) not null,  
    cat_id int not null,  
    FOREIGN KEY fk_cat(cat_id)  
    REFERENCES categories(cat_id)  
)
```

OTHER CONSTRAINTS

- **DEFAULT:** Specifies a value for a column, when an insert statement does not include a value for the column
- **CHECK:** Ensures a value for a column falls within a list of specified values

TRANSACTIONS

- A set of SQL statements that ALL succeed, or ALL fail (atomic)
- Useful when some operation involves multiple DB operations that must happen atomically
 - Eg. Fund transfer in a bank involves simultaneous credit and debit of 2 accounts
- Transactions can involve only **DML commands**
 - Eg. TRUNCATE cannot be part of transaction, but DELETE * can
- You **mark the beginning of a transaction**, and then either **commit** or **rollback** at the end of the set of SQL queries that are to be executed together.
- In addition, you can have intermediate steps marked using as **savepoints** (where you can rollback to)

Reference: <https://www.geeksforgeeks.org/sql-transactions/>

DB ADMINISTRATION

- A set of SQL statements that ALL succeed, or ALL fail (atomic)
- Useful when some operation involves multiple DB operations that must happen atomically
 - Eg. Fund transfer in a bank involves simultaneous credit and debit of 2 accounts
- Transactions can involve only **DML commands**
 - Eg. TRUNCATE cannot be part of transaction, but DELETE * can
- You **mark the beginning of a transaction**, and then either **commit** or **rollback** at the end of the set of SQL queries that are to be executed together.
- In addition, you can have intermediate steps marked using as **savepoints** (where you can rollback to)

Reference: <https://www.geeksforgeeks.org/sql-transactions/>

REFERENCES

- MySQL site: <https://dev.mysql.com/>
- MySQL Official documentation: <https://dev.mysql.com/doc/refman/8.0/en/>
- Tutorials on MySQL
 - w3schools.com: <https://www.w3schools.com/SQL/>
 - MySQL Tutorials: <http://www.mysqltutorial.org/>
- Comparison of SQL syntax across various popular RDBMSes
 - <http://troels.arvin.dk/db/rdbms/>



THANK YOU!

COPYRIGHT NOTICE

© PRASHANTH PURANIK, WWW.DIGDEEPER.IN

All rights reserved. No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the author, except in the case of brief quotations embodied in critical reviews and certain other non-commercial uses permitted by copyright law. For permission requests, write to the publisher, addressed "Attention:Permissions Coordinator" at the address below.

Email: puranik@digdeeper.in

Website: www.digdeeper.in